Implementing session timeouts in a React/Next.js application involves a combination of **client-side inactivity tracking** and **server-side session management**. A robust approach uses a library like `react-idle-timer` for client-side monitoring and integrates with your authentication system (e.g., NextAuth.js).

Prerequisites

- An existing React or Next.js project with an authentication system (like NextAuth.js) that manages user sessions and provides a logout function.

Step-by-Step Guide

1. Server-Side Configuration (Session Expiry)

The server is the ultimate source of truth for session validity. Your authentication provider (e.g., NextAuth.js, Auth0, or custom backend) must issue tokens or manage sessions with a defined, absolute expiration time. Stack Overflow +1

- **NextAuth.js Example:** In your `[...nextauth].ts` file, you can set custom expiry times in the `session` callback and add this expiry time to the session object that the client can read.

typescript

```typescript
// pages/api/auth/[...nextauth].ts (or app/api/auth/[...nextauth]/route.ts)
callbacks: {
  async session({ session, token }) {
    // ... existing session logic
    if (session.user) {
      // Set expiration to 1 hour (for example)
      session.user.session_expiry = new Date(Date.now() + 60 * 60 * 1000).toISOString()
    }
    return session;
  },
}
```

Stack Overflow

2. Client-Side Implementation (Inactivity Detection)

Use the `react-idle-timer` library to detect user inactivity on the client side and trigger a logout or a warning modal.

- Install the package:

bash

```
npm install react-idle-timer
# or
yarn add react-idle-timer
```

- Create a `SessionTimeout` component:

javascript

```
// components/SessionTimeout.jsx
import React, { useRef, useState, useEffect } from 'react';
import { useIdleTimer } from 'react-idle-timer';
// Import your logout function from wherever you manage auth (e.g., NextAuth's signOut)
// import { signOut } from 'next-auth/react';

const SessionTimeout = ({ isAuthenticated, logOut }) => {
  const [timeoutModalOpen, setTimeoutModalOpen] = useState(false);
  const [countdown, setCountdown] = useState(0);
  const idleTimerRef = useRef(null);
  let countdownInterval;
  const IDLE_TIMEOUT = 1000 * 60 * 15; // 15 minutes of inactivity
  const PROMPT_TIMEOUT = 1000 * 60 * 2; // Show warning 2 minutes before actual logout

  const handleLogout = () => {
    setTimeoutModalOpen(false);
    clearInterval(countdownInterval);
    logOut(); // Your actual logout function
  };

  const handleContinue = () => {
    setTimeoutModalOpen(false);
    clearInterval(countdownInterval);
    // This will reset the idle timer in the library automatically
  };

  const onPrompt = () => {
    if (isAuthenticated) {
      setTimeoutModalOpen(true);
      setCountdown(PROMPT_TIMEOUT / 1000); // Start countdown in seconds
      countdownInterval = setInterval(() => {
        setCountdown(prevCount => (prevCount > 0 ? prevCount - 1 : 0));
      }, 1000);
    }
  };
```

```
  const onIdle = () => {
    if (isAuthenticated && timeoutModalOpen) {
      handleLogout();
    }
  };

  // Cleanup interval on component unmount
  useEffect(() => {
    return () => clearInterval(countdownInterval);
  }, [countdownInterval]);

  useIdleTimer({
    ref: idleTimerRef,
    timeout: IDLE_TIMEOUT,
    promptTimeout: PROMPT_TIMEOUT,
    onPrompt: onPrompt,
    onIdle: onIdle,
    debounce: 500, // Debounce activity events
    events: ['mousemove', 'mousedown', 'click', 'scroll', 'keypress', 'load'], // Stand
  });

  return (
    <>
      {/* Render your warning modal here, passing open state, countdown, and handlers *
      {/* A standard dialog component (e.g., from Material UI or custom) can be used */
      {timeoutModalOpen && (
          <SessionTimeoutDialog
              open={timeoutModalOpen}
              countdown={countdown}
              onContinue={handleContinue}
              onLogout={handleLogout}
          />
      )}
    </>
  );
};

export default SessionTimeout;
```

3. Integrate into Your Layout

Wrap your main application layout with the `SessionTimeout` component. This ensures the timer runs across all pages.

- **Next.js App Router (in `layout.tsx`):**

tsx

```tsx
// app/layout.tsx
import { AuthProvider } from './providers/auth-provider'; // A context provider for au

export default function RootLayout({ children }) {
  // Assume AuthProvider manages the global isAuthenticated state and logOut function
  return (
    <html lang="en">
      <body>
        <AuthProvider>
          {children}
          {/* Add the SessionTimeout component here */}
          {/* <SessionTimeout isAuthenticated={...} logOut={...} /> */}
        </AuthProvider>
      </body>
    </html>
  );
}
```

4. Additional Security (API Calls Check)

For maximum security, always verify session validity on the server during API calls. If a client-side API call receives a `401 Unauthorized` error, your application should redirect the user to the login page. Auth0 Community +1

- This involves setting up an API interceptor or wrapper that catches `401` responses and triggers the client-side `logOut` function.